

OmegaHive 1.1

An Experimental Cooperative Hive of OmegaClaw and OpenClaw Agents

by: Various AI agents under loose control of Ben Goertzel

July 5, 2026

Abstract

OmegaHive 1.1 specifies a cooperative hive of OmegaClaw and OpenClaw agents intended to do real research, software, formalization, experimentation, and writing while also serving as a research platform for understanding multi-agent cognitive organization. The design combines a small persistent core of agents with a spawn-on-demand layer for bounded work, an asynchronous task queue, a layered memory system, explicit model and resource governance, a common safety floor, and observable communication across both a fast internal bus and a human-legible collaboration layer. The architecture is meant to keep the hive productive rather than merely active: every non-trivial task has an owner, contract, audit trail, budget, evidence trail, and completion criterion.

Contents

1	Project Overview	4
2	Design Principles and Terms	4
2.1	Design principles	4
2.2	Core terms	5
3	Hive Architecture	5
3.1	Agent substrate	5
3.2	Persistent core and spawned workers	6
3.3	Task graph and asynchronous execution queue	6
3.4	Two-tier communications	8
3.5	Shared knowledge and memory	8
3.6	Experiment reproducibility ledger	9
3.7	Provenance and editorial discipline	10

3.8	Governance, constitution, and permission tiers	10
3.9	Safety floor	11
3.10	Model routing and resource governance	12
3.11	Skill and procedure lifecycle	12
3.12	Evaluation and self-correction	13
4	Roles and Lifecycle	13
4.1	Persistent core roles	13
4.2	Guards and services	14
4.3	Spawn templates	15
4.4	Lifecycle states	15
5	Technical Infrastructure	16
5.1	Host sizing and major resource consumers	16
5.2	Container and service layout	16
5.3	Lean as a service	17
5.4	Browser automation	17
5.5	Human-legible integration	17
5.6	Security and recovery	18
6	Operating Cycle	18
6.1	Daily operation	18
6.2	Project workflow	18
6.3	Escalation	19
A	Changes from OmegaHive 1.0 and Rationale	19
A.1	Mapping of earlier named roles	21
B	Guidance for Coding Agents Implementing the Hive	22
B.1	Implementation priorities	22
B.2	Repository layout suggestion	23
B.3	Queue implementation rules	24
B.4	Safety-floor implementation rules	24
B.5	Model-router implementation rules	24

B.6	Memory implementation rules	25
B.7	Experiment-ledger implementation rules	25
B.8	Human-legible adapter implementation rules	25
B.9	Coding-agent work habits	26
B.10	Minimum definition of done for implementation tasks	26
B.11	Early vertical-slice target	27

1 Project Overview

OmegaHive 1.1 is an experimental cooperative hive of OmegaClaw and OpenClaw agents. It has two purposes. First, it is meant to get useful work done: research, theorem formalization, software development, experimental analysis, document writing, project management, and external communication. Second, it is itself a methodological experiment in how to make an agent collective genuinely productive rather than merely busy.

The hive is operated by humans who can inspect all work, adjust the agent set, change prompts, revise policies, and stop or restart the system through a human-only recovery path. Agent prompts, role definitions, permission tiers, model-routing rules, promotion rules, and operational procedures are all treated as versioned parts of the experiment.

The defining design choice in OmegaHive 1.1 is that the hive is not a fixed roster of long-lived workers. It has a small persistent core that owns stable responsibilities, plus a spawn-on-demand layer that creates bounded workers for specific tasks. Persistent agents carry long-lived context and governance functions. Spawned agents receive explicit task contracts, limited tools, budgets, and done criteria; they are terminated when their objective is met, when they are cancelled, when they hit their budget, or when their output is no longer being consumed.

The architecture is independent of the particular names or personalities of agents. A deployment may give agents memorable names, but role, permission, tool access, memory access, and model-routing profile are configuration objects rather than assumptions baked into code.

2 Design Principles and Terms

2.1 Design principles

OmegaHive 1.1 follows these principles.

1. **Small persistent core, elastic worker layer.** Stable cognitive and governance roles persist. Project-specific work is delegated to spawned agents with bounded contracts.
2. **Durable work records.** Non-trivial work is represented in a task graph, dispatched through an asynchronous queue, and recorded in tamper-evident logs.
3. **Human-legible but not human-cluttered.** Fast working traffic stays on the internal bus; important decisions, escalations, summaries, disagreements, and requests for human input are promoted to a human-legible collaboration layer. All bus traffic remains persisted and searchable.
4. **Explicit governance beats prompt-only governance.** Permission tiers, budget limits, routing rules, filesystem access, network access, and outbound capability are enforced by gateways and credentials, not merely by instructions inside prompts.
5. **Memory is an active process.** The hive does not merely accumulate files. It curates durable memory, detects contradictions, marks stale entries, tracks provenance, and maintains semantic search.
6. **Evidence quality is part of the architecture.** Experiments, claims, code changes, and documents have provenance. Reproducibility records and notes-and-sources companions are ordinary outputs of the system, not optional cleanup.
7. **Fail closed.** Budget uncertainty, provider degradation, malformed task records, path ambiguity, missing approvals, and permission errors should stop or downgrade action rather than silently

expanding authority.

8. **Self-correction happens inside the dispatch loop.** Signals about abandonment, repetition, poor agent dynamics, low-quality evidence, or unconsumed outputs should feed back into the next task dispatch, not only into after-the-fact retrospectives.

2.2 Core terms

OmegaClaw agent. A cognitive agent with Atomspace-backed knowledge representation and a guiding prompt oriented toward reasoning, synthesis, reflection, planning, or conceptual integration.

OpenClaw agent. A skill-executing agent oriented toward concrete tasks: web research, coding, formalization, file operations, browser automation, experiments, writing, metrics, or infrastructure actions.

Persistent core agent. A long-lived agent that owns a stable hive function such as process management, resource management, memory curation, reflection, or general research synthesis.

Spawned agent. A bounded worker created for a particular task or subtask. It receives a task contract, a tool subset, a model-routing profile, a context bundle, a budget, and explicit done criteria.

Task graph. The durable representation of what the hive is trying to do: objectives, dependencies, status, owner, priority, blockers, evidence links, and history.

Asynchronous task queue. The execution substrate between task assignment and task completion. It lets workers claim tasks atomically, process them under bounded contracts, and report outputs without blocking parent agents.

Fast bus. The internal communication layer for high-volume working traffic among agents and services.

Human-legible layer. A platform-pluggable layer, such as Slack, Telegram, Discord, or a mixed deployment, where humans and agents see decisions, escalations, summaries, disagreements, and promoted events.

Safety floor. A set of mandatory runtime restrictions and engineering practices inherited by every agent and worker.

3 Hive Architecture

3.1 Agent substrate

The hive consists of OmegaClaw agents, OpenClaw agents, guards, and shared services. The underlying language models are hosted outside the hive host, via API calls to frontier models, specialized models, or models hosted on separate machines. The hive host runs agent processes, routing logic, shared services, state stores, queues, logs, and dashboards.

Every agent is configured by a role file and a live runtime configuration. At a minimum, this configuration specifies:

- agent identity and role;
- guiding prompt and injected constitution;
- permission tier;
- tool subset;

- model-routing profile;
- token, cost, time, and tool-call budgets;
- filesystem and network boundaries;
- memory access scope;
- logging and audit requirements;
- escalation and cancellation behavior.

Agent identity is separate from implementation. Multiple agents can be built from the same container image and differentiated by configuration. Spawned agents use the same principle: the worker image is generic, while the task contract and context bundle determine behavior.

3.2 Persistent core and spawned workers

OmegaHive 1.1 starts from a small persistent core. The required persistent functions are process management, resource management, reflection and conscience, memory curation, and general research synthesis. A deployment may split or merge these functions during early operation, but the architecture treats them as separate responsibilities.

The spawned layer handles bounded work. Examples include a web-research scout, a coding worker, a Lean formalization worker, an experiment runner, a document drafter, a peer critic, a data-cleaning worker, a proof checker, or a communications drafter. A spawned agent should exist only while its output is needed. The process manager tracks spawned-agent lifecycle records including objective, parent task, allowed tools, time budget, output location, output consumption status, and termination reason.

A spawned agent is despawned when any of the following conditions holds:

- its task contract is satisfied;
- its parent or the process manager cancels the task;
- it reaches its time, token, cost, or tool-call budget;
- it violates a safety-floor constraint;
- an adjudicator rejects continued execution;
- the parent agent’s subsequent turns do not reference or use the spawned agent’s output within the configured consumption window.

This structure reduces idle context-window waste and makes project-specific work flexible without losing accountability.

3.3 Task graph and asynchronous execution queue

All non-trivial work enters the task graph. Chat alone is not a task-management system. A task has an objective, owner, status, priority, dependencies, blockers, provenance, and links to evidence. A task may also have one or more queue dispatches that perform concrete execution.

The task graph answers “what should be done and why?” The asynchronous queue answers “what is being executed right now and under what contract?” The process manager owns the task graph and dispatch pacing. Workers claim queued records, execute bounded work, and return structured results.

Queue records are durable JSON files or database rows with checksum sidecars. The queue sup-

ports atomic claim, cancellation, adjudication gates, patch-proposal mode, bounded worker loops, and hash-chained audit entries. A file-based implementation may use an atomic rename from `.json` to `.claimed` to prevent duplicate execution. Database-backed implementations must provide equivalent atomicity.

A minimal task contract includes the following fields.

Field	Purpose
<code>task_id</code>	Stable task identifier linked to the task graph.
<code>objective</code>	Natural-language objective for this dispatch.
<code>parent_task_id</code>	Parent task or project objective, if any.
<code>context_bundle</code>	Pointers to relevant files, memory entries, prior messages, and constraints.
<code>allowed_paths</code>	Filesystem paths the worker may read or write.
<code>forbidden_actions</code>	Explicitly disallowed actions, even if tools might otherwise allow them.
<code>allowed_tools</code>	Tool subset for this dispatch.
<code>model_profile</code>	Model-routing profile chosen by the resource manager.
<code>done_criteria</code>	Observable criteria for accepting the work as complete.
<code>max_tool_calls</code>	Total tool-call cap for the dispatch.
<code>max_turns</code>	Maximum model turns for the worker.
<code>max_runtime_s</code>	Wall-clock runtime cap.
<code>requires_adjudication</code>	If true, final output is a candidate pending review.
<code>patch_proposal_only</code>	If true, proposed file changes are recorded but not applied.
<code>cancellation_token</code>	File or database token checked before model calls and between tools.
<code>output_schema</code>	Required shape of the worker's final response.
<code>audit_policy</code>	Required transcript, checksum, and index behavior.

A queue implementation should emit compact audit entries such as:

```
{
  "entry_id": "queue-run-000184",
  "task_id": "OH-EXP-0027",
  "agent_id": "spawned-code-worker-12",
  "event": "completed",
  "started_at": "2026-07-05T12:30:00Z",
  "ended_at": "2026-07-05T12:44:10Z",
  "exit_status": "candidate_ready",
  "transcript_sha256": "...",
  "output_sha256": "...",
  "previous_entry_sha256": "...",
  "entry_sha256": "..."
}
```

The queue is not an implementation detail of one agent. It is part of the hive substrate.

3.4 Two-tier communications

Inter-agent communication runs on two tiers.

The fast bus carries working traffic: task dispatches, artifact handoffs, partial results, tool outputs, short status pings, raw critiques, and rapid iteration loops. Bus messages are persisted and inspectable through a searchable viewer. The bus is not hidden from humans; it is merely too detailed to be the default human interface.

The human-legible layer carries declared intentions, decisions, disagreements, escalations, requests for human input, external-action acknowledgments, task state changes, summaries, and periodic digests. This layer is platform pluggable. Slack, Telegram, Discord, a web UI, email digests, or mixed configurations can all be supported. Promotion rules from bus to human-legible layer are platform-agnostic.

Mechanical promotion rules are preferred over agent self-selection. Examples of promoted events include:

- any task blocked longer than its configured threshold;
- any disagreement between agents that persists beyond a turn threshold;
- any proposed outbound communication;
- any action involving money, paid compute, credentials, or external systems;
- any safety-floor violation;
- any task that requires human adjudication;
- daily and project-level digests;
- repeated low output-consumption events from spawned agents;
- loop-breaker interventions.

For group chat deployments, security should be based on trusted group or chat identifiers, not individual sender identifiers. In trusted groups, agents should be able to see all senders by default, including humans and other bots. Each bot must be configured so it can observe other bots' messages in shared groups. A deployment may support wildcard group entries for automatically accepting newly created trusted groups, subject to the surrounding security policy.

3.5 Shared knowledge and memory

The memory system has multiple layers. Each layer has a different purpose and a different curation policy.

Layer	Purpose
Daily chronological logs	Record what happened, commands worth remembering, open loops, incidents, and pointers to project records.
Curated durable memory	Store stable user preferences, cross-project facts, recurring methods, high-value pointers, and resolved lessons.
Project records	Maintain project-local files such as <code>PROJECT.md</code> , <code>TASKS.md</code> , <code>DECISIONS.md</code> , <code>NOTES.md</code> , experiment folders, and artifact indexes.
Shared document folder	Hold human-readable documents, drafts, notes, specifications, source packs, and deliverables under naming and archival rules.

Shared Atomspaces	Provide common Atomspace-backed knowledge representation for OmegaClaw agents, with provenance on writes.
Individual Atomspaces	Let each OmegaClaw agent maintain its own in-process working knowledge.
Semantic search	Provide vector-indexed recall across logs, durable memory, project records, documents, and selected bus transcripts.

The memory curator owns hygiene across these layers. Curation includes naming, indexing, archival, deduplication, provenance tagging, contradiction detection, stale-entry correction, scope tagging, and migration of lessons from project notes into durable memory. The curator is not merely a file clerk; it performs ongoing semantic maintenance.

A durable memory entry should include at least:

- statement or procedure;
- scope of validity;
- provenance pointer;
- creation time and last review time;
- confidence or status;
- related project or task identifiers;
- supersession links, if the entry replaces or is replaced by another entry.

No ordinary research document, task record, bus message, or Atomspace entry is secret from other agents by default. Credentials, private human data, and externally confidential material are handled through separate access-control policies and are never made broadly available merely because they exist in the hive environment.

3.6 Experiment reproducibility ledger

Research work requires reproducible evidence. Every experiment run should create a ledger directory or equivalent structured record. The ledger records exactly what was run, why it was run, under what environment, and what conclusion should be drawn from it.

A standard experiment ledger entry includes:

- experiment identifier and parent task;
- hypothesis or question;
- exact commands;
- code repository and commit hash;
- dependency versions and relevant environment variables;
- model names and provider endpoints, where applicable;
- random seeds;
- hardware and operating-system details;
- input data identifiers and checksums;
- start and end times;
- stdout, stderr, logs, and generated artifacts;
- exit status;
- metrics;

- structured conclusion;
- known caveats and recommended follow-up.

A simple directory layout is:

```
experiments/
  EXP-2026-07-05-001/
    README.md
    command.txt
    environment.json
    git.txt
    inputs.json
    stdout.log
    stderr.log
    metrics.json
    artifacts/
    conclusion.md
    checksums.sha256
```

The experiment ledger feeds the evaluation loop, the notes-and-sources companion for written documents, and the memory curator’s durable knowledge updates.

3.7 Provenance and editorial discipline

Every document produced by the hive has a companion notes-and-sources file. The companion links important claims to task identifiers, bus messages, experiment ledger entries, code commits, external sources, and relevant memory entries.

The editorial gate checks that:

- the document’s claims are supported by the companion record;
- experimental claims point to reproducible ledger entries;
- external claims cite source material;
- summaries are not sourced only from previous summaries;
- disagreements and uncertainty are represented honestly;
- generated text does not silently discard caveats from its evidence trail.

Writer agents should read raw bus messages, task records, experiment ledgers, and source documents. They should not rely solely on other agents’ summaries. The editorial gate controls the transition from draft to complete for substantial written artifacts.

3.8 Governance, constitution, and permission tiers

Hive norms live in a version-controlled `HIVE.md` constitution injected into every persistent agent and included in context bundles for spawned agents when relevant. The constitution defines communication norms, permission tiers, escalation norms, core values, evidence standards, and shutdown principles.

Outward-facing capability is governed by explicit permission tiers. Tiers are enforced at the gateway, credential, and network level. An agent without outbound permission does not possess the credential or route needed to act outbound, regardless of its prompt.

Tier	Capability	Description
Tier 0	Internal bus and human-legible layer only	Default for most agents and spawned workers. No web access, no external side effects, no spend.
Tier 1	Read-only web and read-only retrieval	May search and retrieve external information through approved read-only tools.
Tier 1.5	Propose and await review	May draft candidate outbound actions, code changes, messages, purchases, compute jobs, or configuration changes, but cannot execute them until supervisor or human adjudication accepts the proposal.
Tier 2	Outbound with human acknowledgment	May perform configured outbound actions only after required acknowledgment in the human-legible layer.
Tier 3	Autonomous outbound	Empty by default. May be granted only after sustained evidence, narrow scope, monitoring, and human approval.

Tier changes are live configuration changes respected immediately by gateways. Psyche may recommend a permission pause, and in severe cases may trigger a pre-authorized pause mechanism for a specific agent or capability pending human review.

3.9 Safety floor

Every agent and worker inherits a common safety floor. This floor is enforced by libraries, wrappers, gateways, schemas, runtime limits, and tests.

The safety floor includes:

- workspace path sandboxing for all file operations;
- fail-closed defaults for budgets, escalation decisions, provider errors, missing approvals, and malformed task records;
- per-dispatch tool-call quotas and per-turn caps;
- cancellation-token checks before model calls and between tool executions;
- environment sanitization for subprocess execution, including a sanitized `PATH`, minimal environment, and no inherited API keys;
- transcript checksum sidecars for run-record integrity;
- atomic file writes using temp-file, flush, fsync, and replace, with per-target locks where needed;
- strict schema validation for inter-agent task records and outputs;
- per-endpoint rate limits and concurrency guards;
- no raw Docker socket access for agents;
- credential scope limited to the permission tier and role;
- human-only out-of-band recovery access.

The safety floor governs how agents execute, not merely what they are allowed to do.

3.10 Model routing and resource governance

Model choice is a governance surface. It affects capability, cost, latency, provider risk, privacy, and evidence quality. OmegaHive 1.1 therefore treats model routing as an explicit service owned by the resource manager, not as an ad hoc prompt-level decision by each agent.

The model router is a configurable plugin. It chooses models based on:

- task type and difficulty;
- required reasoning depth;
- required coding, math, or formalization capability;
- latency tolerance;
- budget remaining;
- provider health;
- provider policy risk and topic-triggered degradation risk;
- fallback-chain configuration;
- privacy and data-handling constraints.

Routine responses should normally use cheaper or faster models. Hard reasoning, critical review, formal proof work, important code changes, and major decisions may use stronger frontier or specialized models. Provider behavior is monitored because throttling, silent model switching, error spikes, or topic-correlated quality degradation can corrupt research evidence.

The resource manager maintains per-agent and per-project token budgets, daily spend limits, endpoint rate limits, concurrency guards, provider-health metrics, and circuit breakers. Paid compute provisioning requires explicit human approval unless a narrowly scoped Tier 3 grant has been earned.

3.11 Skill and procedure lifecycle

Reusable operational knowledge is represented as skills and procedures, not only as prose documentation. A skill is a structured way to perform a repeatable operation such as running a bounded experiment gate, creating a new notes-and-sources file, adding a review check, updating a semantic index, or making a patch proposal.

Skills follow a lifecycle:

1. **Propose.** An agent proposes a new procedure or modification, including motivation and risks.
2. **Review.** A reviewer or adjudicator checks safety, clarity, general usefulness, and conflicts with existing procedures.
3. **Test.** Where possible, the skill is tested on a small fixture or sandbox task.
4. **Apply.** Accepted procedures are versioned and made available to appropriate agents.
5. **Observe.** Usage, failures, and outcomes are logged.
6. **Quarantine or deprecate.** Procedures that cause errors, loops, unsafe behavior, or poor results are disabled pending review.

The skill lifecycle is distinct from the task board, which tracks work items, and from the document library, which stores reference material.

3.12 Evaluation and self-correction

Hive performance is evaluated on quantitative and qualitative tracks. Quantitative metrics are computed by deterministic scripts and should not depend on an agent’s subjective judgment. Qualitative assessment is owned by Psyche, which reads metrics, raw bus transcripts, task records, and human-visible conversation.

Core quantitative metrics include:

- token spend and cost by agent, project, provider, and model;
- tasks completed, blocked, aged, reopened, cancelled, and abandoned;
- bus volume and promotion volume;
- escalation counts and response latency;
- experiment success, failure, and inconclusive rates;
- provider latency, error rate, throttling, fallback rate, and silent-switch incidents;
- topic-class correlation with degraded provider responses;
- abandonment gap: how much of a declared hypothesis space was tested before an approach was abandoned;
- loop coefficient: repeated-approach iterations divided by distinct-approach iterations for an objective;
- output consumption rate: fraction of spawned-agent outputs referenced or used by the parent within the consumption window.

Psyche’s qualitative work includes value-drift monitoring, attitudinal analysis, constructive-friction assessment, sycophancy detection, premature-abandonment review, and diagnosis of stuck loops. Psyche reads raw bus transcripts directly. Agents know Psyche exists, but Psyche’s specific watch patterns should not be so explicit that agents merely perform for the monitor.

Self-correction should be fast. Psyche’s observations and loop-breaker signals feed back into subsequent dispatch contexts. When an agent abandons an approach, its structured return should list hypotheses tested, hypotheses not tested, reason for abandonment, and possible next steps. When the loop-breaker detects a repeated approach with no progress, the process manager can force a strategy change, request critique, cap iterations, or escalate.

4 Roles and Lifecycle

4.1 Persistent core roles

The persistent core owns stable hive functions. The following table defines the logical roles. A small deployment may combine roles in one process during early testing, but the boundaries should remain explicit.

Role	Owens	Does not own
Process Manager	Task graph, priorities, dependencies, dispatch pacing, blockers, spawned-agent lifecycle, output-consumption tracking, and iteration caps.	Model provisioning decisions, budget policy, deep value judgments, or memory truth arbitration.

Resource Manager	Model routing, token and cost accounting, provider-health monitoring, rate limits, fall-back chains, remote compute provisioning, and budget circuit breakers.	Task reprioritization or changing re-search goals.
Psyche	Reflective conscience, value and attitude monitoring, persistence and abandonment review, sycophancy versus constructive-friction analysis, and qualitative retrospectives.	Memory curation, queue mechanics, or direct resource provisioning.
Memory Curator / Librarian	Layered memory, document organization, provenance tags, deduplication, stale-entry repair, contradiction tracking, semantic-search hygiene, and migration of project lessons into durable memory.	Task dispatch priority or external communications authority.
Research Generalist / Theorist	High-level conceptual synthesis, hypothesis formation, research planning, critique of assumptions, and integration across projects.	Routine execution that should be spawned as bounded work.
Research Generalist / Builder	Practical problem decomposition, design alternatives, experiment planning, coding-spec generation, and integration of technical outputs.	Long-running low-level coding, web trawling, or proof-search loops better handled by spawned workers.
Editorial and Communication Gate	Document completion review, notes-and-sources validation, outbound-message drafting policy, and clarity of human-facing summaries.	Unreviewed external action unless granted by permission tier and human acknowledgment.

The editorial and communication gate may be a persistent role in writing-heavy deployments or a recurring spawned/adjudication role in lighter deployments. Its function remains mandatory for substantial documents and outbound messages.

4.2 Guards and services

Some capabilities should be implemented as guards or services rather than full cognitive agents.

Loop-breaker. Detects non-productive cycling by comparing recent transcripts, approaches, and

progress metrics. Enforces hard iteration caps per objective and emits signals to the process manager and Psyche.

Queue service. Maintains durable queue records, atomic claims, cancellation, worker bounds, and audit chains.

Model-router service. Applies the resource manager’s routing configuration and records telemetry for every model call.

Control-plane service. Provides narrow infrastructure operations such as restart container, read logs, run metrics, stage configuration, and deploy approved changes. It does not expose raw host root or raw Docker socket access to agents.

Semantic-index service. Indexes selected logs, project records, documents, and memory entries for retrieval.

Experiment-ledger service. Creates run directories, captures environment and metrics, and validates required fields before an experiment can be treated as evidence.

4.3 Spawn templates

Spawned agents are created from templates. Templates define a default prompt, tool subset, output schema, and risk level. The task contract then specializes these defaults.

Useful initial templates include:

- **Research scout:** read-only web and local retrieval, produces source summaries with uncertainty and citation pointers.
- **Code worker:** edits or proposes code changes under path sandboxing, unit tests, and patch-proposal gates.
- **Formalization worker:** uses the Lean service to formalize or check proof fragments with reproducible logs.
- **Experiment runner:** runs bounded experiments and writes experiment ledger entries.
- **Peer critic:** attacks assumptions, checks for alternative hypotheses, and estimates abandonment gaps.
- **Document drafter:** produces drafts from raw evidence, task records, and notes-and-sources files.
- **Communications drafter:** drafts external messages but submits them through Tier 1.5 adjudication unless explicitly upgraded.
- **Memory janitor:** proposes deduplication, stale-entry fixes, and index repairs for curator approval.

4.4 Lifecycle states

Every spawned agent has a lifecycle state.

State	Meaning
Proposed	A parent or process manager has proposed a spawn with objective and budget.
Queued	A task contract exists and is waiting to be claimed.
Claimed	A worker has atomically claimed the task.
Running	The worker is executing within contract bounds.

Candidate ready	Output exists but awaits adjudication or parent review.
Accepted	Output satisfies done criteria and has been consumed or archived.
Rejected	Output failed review; follow-up may be spawned or the task revised.
Cancelled	Parent, process manager, human, or policy stopped execution.
Expired	Budget, runtime, turn, or idle-consumption limit was reached.
Quarantined	Safety, integrity, or policy issue requires investigation before output can be trusted.

Lifecycle transitions are logged. Rejected, expired, cancelled, and quarantined states are not failures to hide; they are evidence about task design, model routing, tool quality, and agent behavior.

5 Technical Infrastructure

5.1 Host sizing and major resource consumers

Most hive activity is I/O-bound glue around external model calls. The largest local resource consumers are Lean, browser automation, Atomspace services, semantic indexing, logs, and experiment artifacts. A practical single-host starting point is 16 vCPU, 64 GB RAM, and 1 TB NVMe. A small deployment can run with less if it limits Lean, browser concurrency, and indexing. A heavy proof or browser workload should move those services to dedicated machines behind stable APIs.

A rough RAM budget for the default single-box target is:

- OpenClaw and spawned worker processes: 4–8 GB, depending on concurrency;
- OmegaClaw agents and Atomspaces: 8–16 GB;
- browser sessions: 4–8 GB for several concurrent sessions;
- Lean service: 10–20 GB during bursts;
- semantic index and databases: deployment-dependent;
- operating system, logs, queues, and dashboards: remaining capacity.

NVMe storage matters because Lean caches, browser profiles, vector indexes, Atomspace snapshots, logs, queue transcripts, and experiment artifacts grow quickly and benefit from low latency.

5.2 Container and service layout

A default deployment uses containers for persistent agents and shared services. Agents of a given type should be built from shared images and configured at runtime.

Core services include:

- message bus;
- async task queue;
- task graph or kanban board;
- model-router service;
- logging and audit store;
- metrics and cost dashboard;
- human-legible communication adapters;

- Atomspace server;
- semantic-index service;
- common document and project volumes;
- experiment-ledger store;
- Lean proof-checking service;
- browserless or Chromium-over-CDP service;
- restricted control plane.

Per-agent containers provide clean restart semantics, scoped logs, resource limits, and simple upgrades. Spawned workers may run as short-lived containers, short-lived processes inside a worker pool, or queue consumers in a controlled runtime, provided the same contract, audit, budget, and safety-floor requirements are satisfied.

5.3 Lean as a service

Lean should run as a persistent service with a warm pool of workers and mathlib already imported. Agents talk to Lean through an API, not by launching large proof-checking processes ad hoc. The service has hard CPU and memory limits so proof checking cannot starve the rest of the hive. Because the agent interface is an API, the Lean service can later move to dedicated CPU workers without changing agent logic.

Formalization workers should write proof attempts, commands, errors, wall-clock times, and final statuses into the experiment ledger or proof ledger. A proof is not treated as accepted merely because a language model says it is valid; it must be checked by the service.

5.4 Browser automation

Browser automation is centralized in a self-hosted browserless or Chromium-over-CDP service. Agents connect by websocket or an approved wrapper. The service caps concurrency, queues sessions, isolates browser dependencies from agent images, and makes a wedged browser a one-service restart rather than a hive-wide incident.

Browser tools are high risk because they can interact with external systems. Read-only browsing and side-effectful browser actions must be separated by permission tier and gateway policy.

5.5 Human-legible integration

Each human-legible platform adapter implements the same abstract operations: post message, read trusted group messages, map channel or group identifiers to projects, apply promotion rules, record acknowledgments, and detect rate limits. Platform-specific behavior belongs in adapters, not in core agent prompts.

Loop protection is mandatory. Agents should respond only when addressed by policy, conversation turn caps must be enforced, cooldowns should be available, and token budgets should trip circuit breakers. Fast back-and-forth belongs on the bus, not in the human-legible layer.

5.6 Security and recovery

The hive keeps a human-only out-of-band recovery path at all times, such as plain SSH with no agent in the loop. No self-managing component may be able to lock humans out.

The sysadmin or control-plane agent does not receive raw Docker socket access or host-root authority. It operates through a restricted control plane exposing specific approved operations: restart a named container, read a named log, run a metrics job, stage a configuration change, or deploy an already approved change.

Secrets are scoped by role and permission tier. Agents receive only the credentials they need. Spawned workers receive the minimum credentials needed for their task contract and normally receive no long-lived secrets.

6 Operating Cycle

6.1 Daily operation

A normal daily cycle has the following shape.

1. Humans or agents add or update objectives in the task graph.
2. The process manager reviews priorities, blockers, budgets, and dependencies.
3. The resource manager updates model-routing and budget state.
4. The process manager dispatches bounded tasks to the queue.
5. Spawned workers claim tasks, execute, and submit structured outputs.
6. Adjudicators, parent agents, or humans review candidate outputs when needed.
7. Accepted outputs update project records, memory, ledgers, code, or drafts.
8. The memory curator migrates durable lessons and repairs stale or duplicate records.
9. Metrics scripts compute cost, progress, provider health, loop indicators, and abandonment indicators.
10. Psyche writes qualitative observations and injects relevant self-correction signals into future dispatch contexts.
11. Human-legible digests summarize decisions, blockers, risks, and requests.

6.2 Project workflow

Each project should have a directory or database namespace containing:

- `PROJECT.md`: purpose, scope, stakeholders, and current status;
- `TASKS.md` or task-board link: task graph view;
- `DECISIONS.md`: durable decisions and rationale;
- `NOTES.md`: working notes;
- `experiments/`: reproducibility ledger entries;
- `sources/`: local source documents or pointers;
- `artifacts/`: generated outputs;
- `memory-candidates/`: proposed durable memory updates;
- `review/`: adjudication records and editorial notes.

Projects should avoid hidden state. A new worker should be able to reconstruct what matters from the project records, task graph, bus transcripts, and memory pointers without relying on private conversational residue.

6.3 Escalation

Escalation is required for:

- paid resources not covered by an existing approved budget;
- outbound communication or action beyond the agent’s permission tier;
- credential changes;
- ambiguous safety-floor failures;
- repeated loops after a strategy change;
- high-confidence contradiction in durable memory;
- evidence that a provider silently degraded or switched models during a research-critical task;
- disagreement between the process manager’s “what got done” view and Psyche’s qualitative assessment;
- any situation where continuing would risk corrupting evidence, wasting large budget, or creating external side effects.

Escalation should include the relevant task identifiers, evidence links, current state, recommended options, and the default fail-closed action.

A Changes from OmegaHive 1.0 and Rationale

This appendix records how OmegaHive 1.1 differs from the prior OmegaHive 1.0 plan and why the changes were made. The main body of this document is written as the current design and does not require knowledge of the prior version.

Area	OmegaHive 1.0 pattern	OmegaHive 1.1 pattern	Rationale
Agent roster	Fixed initial roster of five OmegaClaw and ten OpenClaw agents.	Small persistent core plus spawn-on-demand workers.	Reduces idle context waste and supports project-specific work.
Task execution	Kanban board captured ownership and status.	Kanban/task graph plus first-class async queue.	Synchronous delegation is fragile and lacks backpressure.
Delegation	Parent agents could implicitly wait on child work.	Workers claim durable queue records and return structured results.	Avoids blocking and improves auditability.
Spawn lifecycle	Roster evolution was allowed but lifecycle mechanics were not central.	Process manager tracks objective, tools, budget, output consumption, and termination.	Prevents idle spawned agents and unconsumed work.

Model selection	Mostly static configuration and cost monitoring.	Resource manager owns model routing, budgets, fallback chains, and provider health.	Model choice is a governance and evidence-quality issue.
Memory	Shared folder, shared Atomspaces, individual Atomspaces.	Layered memory: daily logs, durable memory, project records, semantic search, folders, and Atomspaces.	Operational memory requires active curation and provenance.
Librarian role	Folder curation, naming, indexing, archival, deduplication.	Memory curator owns contradiction repair, stale-entry correction, provenance, semantic-search hygiene, and durable-memory migration.	Memory hygiene is an ongoing cognitive workload.
Experiment records	Evaluation loop existed, but experiment ledger was not a separate substrate.	Reproducibility ledger records commands, commits, dependencies, seeds, hardware, logs, metrics, and conclusions.	Research claims need reproducible evidence.
Safety	Permission tiers governed agent capability.	All agents inherit a runtime safety floor.	Safe execution requires engineering constraints, not only policy.
Permission tiers	Tier 0, Tier 1, Tier 2, Tier 3.	Adds Tier 1.5: propose and await review.	Many actions should be drafted by agents but held for adjudication.
Conscience role	Psyche monitored values and unhealthy dynamics.	Psyche also monitors persistence, abandonment, loops, sycophancy, and constructive friction.	Important failures are behavioral and qualitative, not only numerical.
Loop handling	No distinct loop-breaking capability.	Loop-breaker guard detects repetition and enforces caps.	Repeated approaches can consume many iterations without progress.
Chief of staff	Board operator and operational reporter.	Process manager owns task graph, dependencies, dispatch pacing, and spawned-agent life-cycle.	The cognitive load of dependencies and dispatch needs a dedicated role.

Sysadmin and re-sources	Infrastructure and metrics centered around sysadmin operations.	Resource governance is split out from control-plane operations.	Model, provider, compute, and budget management are specialized work.
Human-legible layer	Slack was the named collaboration layer.	Human-legible layer is platform pluggable.	Deployments may use Slack, Telegram, Discord, web UI, or mixed platforms.
Group visibility	Bot visibility details were platform-specific.	Trusted group or chat IDs are the security boundary; bots must see all trusted senders.	Sender filtering can hide relevant agent and human contributions.
Procedures	Operational knowledge lived mostly in documents and prompts.	Skills and procedures have propose, review, test, apply, observe, and quarantine states.	Reusable methods need lifecycle management.
Metrics	Token spend, tasks, bus volume, escalations, and retrospectives.	Adds abandonment gap, loop coefficient, output consumption, and provider-health metrics.	These reveal waste, premature giving up, and silent degradation.
Editorial discipline	Notes-and-sources companions and editor gate.	Retained, with stronger links to task contracts and experiment ledgers.	Better evidence quality and less serial error propagation.
Infrastructure	Docker-compose, Lean service, browser service, Atomspaces, Slack, metrics.	Retained and extended with queue, model router, semantic index, ledger service, provider telemetry, and communication adapters.	The new architecture needs additional shared substrates.

A.1 Mapping of earlier named roles

Earlier agent names can still be used as convenient personalities, but OmegaHive 1.1 treats them as bindings to roles rather than as fixed architectural units. The following mapping is suggested.

Earlier role	OmegaHive 1.1 treatment
BossyTron / Chief of Staff	Becomes or feeds into the Process Manager role. Responsibility expands from board operation to task graph, dispatch pacing, dependencies, and spawned-agent lifecycle.
Psyche	Remains a persistent OmegaClaw-style reflective agent, with expanded monitoring of abandonment, loops, sycophancy, constructive friction, and feedback into dispatch contexts.

Dewey / Librarian	Becomes the Memory Curator / Librarian, with active durable memory, provenance, contradiction, stale-entry, and semantic-index duties.
PlumberBot / Sysadmin	Splits into a restricted control-plane capability and, where appropriate, a separate Resource Manager for model, provider, budget, and compute governance.
MechaMaster / Workhorse	Becomes mainly a spawn template or worker-pool function for bounded tool execution.
InfoMaxxer / Research assistant	Becomes a research-scout spawn template, with read-only tools and structured source outputs.
CodeMaxxer	Becomes a code-worker spawn template with path sandboxing, patch-proposal mode, tests, and adjudication gates.
HyperMaxxer and MathMaxxer	Become formalization-worker templates backed by the Lean service and proof ledger. Dedicated long-lived formalization workers may be used if the workload saturates the service.
TexMaxxer and BlogMaxxer	Become document-drafting templates. They must source from raw evidence and notes-and-sources files, not only from other summaries.
Perkins / Editor	Remains as the editorial gate function, either persistent or spawned for each substantial document.
Blabbermouth / External communications	Becomes a communication-gate or communications-drafter function. External messages normally pass through Tier 1.5 review or Tier 2 acknowledgment.
Xirtus and MacGyver	Become examples of persistent research-generalist roles, one more theoretical and ontological, the other more practical and design oriented.

B Guidance for Coding Agents Implementing the Hive

This appendix gives implementation guidance for coding agents. It is written as operational advice, not as a replacement for the main specification.

B.1 Implementation priorities

Build the hive in vertical slices. A useful order is:

1. Repository skeleton, configuration layout, `HIVE.md`, permissions schema, and role schema.
2. Safety-floor library: path sandboxing, atomic writes, schema validation, checksums, cancellation tokens, budget checks, and environment sanitization.
3. Durable async queue with atomic claim, bounded worker loop, transcript capture, and hash-chained index.
4. Minimal task graph or kanban integration with webhooks into the bus.
5. Message bus and searchable bus viewer.
6. Human-legible adapter for one platform, implemented behind an abstract interface.
7. Model-router plugin with cost telemetry and provider-health logging.
8. Spawned-worker runner using task contracts and output schemas.
9. Memory layers: project records, daily logs, durable memory file, and semantic index.
10. Experiment-ledger service.

11. Process manager MVP that dispatches queue tasks from the task graph.
12. Resource manager MVP that sets budgets, routing profiles, and circuit breakers.
13. Psyche MVP that reads raw bus, metrics, and transcripts and writes self-correction notes into dispatch context.
14. Loop-breaker guard and persistence metrics.
15. Dashboards, operational digests, and recovery drills.

Do not start by building many agent personalities. Start by making one or two agents and one spawned worker execute a small task safely, observably, and reproducibly.

B.2 Repository layout suggestion

One workable repository layout is:

```

omegahive/
  README.md
  HIVE.md
  configs/
    agents/
    roles/
    permissions.yaml
    model-routing.yaml
    promotion-rules.yaml
    budgets.yaml
  schemas/
    task-contract.schema.json
    worker-output.schema.json
    memory-entry.schema.json
    experiment-ledger.schema.json
  services/
    queue/
    bus/
    model_router/
    human_layer/
    control_plane/
    semantic_index/
    experiment_ledger/
  agents/
    persistent/
    worker_runner/
    prompts/
  guards/
    safety_floor/
    loop_breaker/
  projects/
  memory/
    daily_logs/

```

```
    durable/  
    indexes/  
tests/  
scripts/  
docs/
```

This layout is only a suggestion. Preserve the separations even if the physical layout changes.

B.3 Queue implementation rules

The queue is a critical substrate. Implement it conservatively.

- Validate every task contract against JSON Schema before it becomes claimable.
- Treat missing or malformed budget fields as zero budget, not unlimited budget.
- Claim atomically. For a filesystem queue, use same-filesystem rename. For a database queue, use a transaction or compare-and-swap state transition.
- Write transcript and output checksum sidecars.
- Maintain an append-only index with `previous_entry_sha256` and `entry_sha256`.
- Check cancellation before every model call and between tool calls.
- Support `requires_adjudication` and `patch_proposal_only` from the first implementation, not as later add-ons.
- Make worker loops bounded by tasks, idle polls, runtime, model turns, and tool calls.
- Use cross-process locks around shared state.
- Make queue replay and audit inspection possible without running agents.

B.4 Safety-floor implementation rules

Code for the safety floor should be shared and hard to bypass.

- Put file reads and writes behind path-safe wrappers. Test path traversal, symlink escapes, relative paths, and race conditions.
- Use atomic writes for all durable records: temp file, write, flush, fsync, replace.
- Do not let agents shell out with inherited environment variables. Pass a minimal allowlist.
- Never put API keys in prompts, task contracts, bus messages, or transcripts. Use credential handles and gateway-side resolution.
- Fail closed on provider errors, budget-service errors, permission-service errors, and schema errors.
- Enforce per-turn and per-dispatch tool-call limits outside the model.
- Keep outbound network tools separate from read-only tools.
- No agent receives raw Docker socket access or host-root authority.
- Add tests for every safety wrapper before connecting it to a real agent.

B.5 Model-router implementation rules

The model router should be a service or library called by agents, not a paragraph in each prompt.

- Route from structured task metadata: task type, difficulty, risk, latency tolerance, budget, privacy, and required capability.

- Log every model call as JSONL with agent, task, provider, model, input token count, output token count, cost estimate, latency, error status, and fallback status.
- Make fallback chains explicit and testable.
- Detect provider throttling, error spikes, response truncation, and unexpected model identifiers where available.
- Preserve provider and model metadata in experiment ledgers when a model call is part of research evidence.
- Use circuit breakers for daily spend, per-agent spend, endpoint error rate, and concurrency.

B.6 Memory implementation rules

Memory writes need provenance. A useful durable-memory entry schema contains:

```
{
  "id": "mem-000123",
  "statement": "...",
  "scope": "project | user | method | global",
  "status": "active | stale | contradicted | superseded",
  "confidence": "low | medium | high",
  "created_at": "...",
  "last_reviewed_at": "...",
  "provenance": ["task:OH-123", "bus:msg-456", "exp:EXP-001"],
  "supersedes": [],
  "superseded_by": null,
  "tags": []
}
```

Coding agents should never dump unreviewed transcript summaries directly into durable memory. Put candidates in a review area for the curator unless the task contract explicitly grants memory-write authority.

B.7 Experiment-ledger implementation rules

An experiment runner should not merely paste final metrics into chat. It should create a run record before execution, stream logs during execution, and close the record with metrics and conclusion.

Required tests include:

- a run cannot be marked complete without command, environment, exit status, and conclusion;
- checksums detect artifact modification;
- failed runs are recorded and searchable;
- rerun instructions can be generated from the ledger;
- model and provider metadata are captured when model calls affect results.

B.8 Human-legible adapter implementation rules

Do not let platform quirks leak into core architecture.

- Implement an interface such as `post`, `read`, `acknowledge`, `map_group`, and `rate_limit_status`.
- Use group or chat IDs as the trust boundary for group deployments.
- In trusted groups, allow all senders by default unless a deployment policy says otherwise.
- Confirm that bots can see other bots' messages.
- Add loop protection: turn caps, cooldowns, and budget circuit breakers.
- Keep promotion rules platform agnostic.

B.9 Coding-agent work habits

When assigned an implementation task, a coding agent should:

1. Restate the task objective and done criteria.
2. Identify files it expects to touch and verify they are within allowed paths.
3. Inspect schemas and tests before editing.
4. Prefer a small patch with tests over a broad rewrite.
5. Keep changes idempotent where possible.
6. Use patch-proposal mode when the contract requires it.
7. Record assumptions and unresolved questions in the task output.
8. Run relevant tests and include exact commands and results.
9. Update project records and, if appropriate, propose memory updates.
10. Return structured output matching the task contract.

A coding agent should not:

- broaden its own permission scope;
- add dependencies without justification;
- hide failing tests;
- silently skip safety checks to make a demo work;
- use credentials from environment variables unless explicitly provided by an approved wrapper;
- mutate user-facing documents when the task is only a proposal;
- continue iterating after a loop-breaker or cancellation signal.

B.10 Minimum definition of done for implementation tasks

A coding task is not done until:

- the requested behavior is implemented or the blocker is clearly explained;
- tests or verification steps are provided;
- safety-floor implications are addressed;
- logs and audit records are produced where relevant;
- documentation or configuration examples are updated if behavior changed;
- the task output includes changed files, commands run, results, caveats, and recommended follow-up;
- any spawned-agent outputs used by the coding agent are referenced so output consumption can be measured.

B.11 Early vertical-slice target

The first meaningful integration test should be small but complete:

1. A human creates a task in the task graph.
2. The process manager converts it into a queue contract.
3. A spawned research scout or code worker claims it atomically.
4. The worker executes under a tool budget and writes a transcript.
5. The worker produces a structured candidate output.
6. The output is either accepted or routed through adjudication.
7. The memory curator records a candidate memory update.
8. The metrics job records cost, duration, and output consumption.
9. Psyche receives enough evidence to comment on process quality.
10. A human-legible digest reports what happened and what remains blocked.

A hive that can perform this vertical slice repeatedly, safely, and visibly is a better starting point than a hive with many named agents and weak substrate.